A REPORT

ON

STUDENT REPORT MANAGEMENT SYSTEM PROJECT

By

Name of the Student	**Registration No.**
Harini. K	AP24110010123

Prepared in the partial fulfillment of the
Project Based Learning of Course CSE 201 – Coding Skill - 1

SRM UNIVERSITY, AP

NOVEMBER 2025

ABSTRACT

This project is a Student Report Management System developed in the C programming language. It uses text files to store student records and includes a login system where users can sign in as an Admin, Staff member, or Guest. Each type of user is given different access rights.

The system keeps basic student details such as roll number, name, and marks. Depending on the user's role, they can perform various actions like adding new records, viewing existing information, searching for a particular student, editing details, or removing entries.

Admins can use all available features, staff members are allowed to view and search data, and guests can only see the student list. This layered permission structure helps protect the data and ensures that only authorized users can make changes to the records.

INTRODUCTION

The Student Report Management System (SRMS) is a C-based project designed to manage student details by using basic file-handling methods. It saves information like roll numbers, names, and marks in a text file so that the data remains stored even after the program is closed. The system also includes a login mechanism with different user roles to ensure that only the right users can access specific features.

The application supports three roles: Admin, Staff, and Guest. Admin users have complete access—they can add new student entries, modify existing data, delete records, search for a particular student, and view all stored information. Staff members are limited to viewing and searching records, while guests can only view the available data.

By dividing access in this structured manner, the system protects the stored information and avoids any unauthorized changes. The project also highlights essential concepts of the C language, such as the use of structures, file processing, string manipulation, and conditional statements. Overall, this system is a simple demonstration of how data can be organized and managed in a real-world application using fundamental C programming techniques.

OBJECTIVES

- To design a basic student information system in C that can reliably handle and organize student data.
- To incorporate a login system that **identifies users by their role** and grants access accordingly.
- To ensure **proper authorization by limiting system** functions based on whether the user is an Admin, Staff member, or Guest.
- To use text files for storing student records so that information remains available across multiple program executions.
- To give Admin-level users full management control, including the ability to add new entries, review data, search for students, modify details, and remove records when necessary.
- To allow Staff users to access only non-editable features such as viewing and searching student information.
- To offer Guests simple viewing access to maintain the confidentiality and safety of stored data.
- To practice the use of essential C programming elements such as structures, file operations, input handling, and conditional checks.
- To safeguard the accuracy and consistency of the data by ensuring only authorized users can alter information.
- To **implement a clear and straightforward menu system** that helps users navigate the application without difficulty.

MODULES

1. Main Program Module:

This module consists of the **main()** function, which serves as the starting point of the program. It initiates the execution by calling the login system to authenticate the user. Once the login is successful, it directs the user to the appropriate menu based on their role, whether Admin, Staff, or Guest. This module does not require any inputs from other modules at the start, and its outputs include the result of the login process along with navigation to the corresponding role-specific menu.

```c
//main function - display login only
int main(){
    if(loginSystem()){
        mainMenu();
    }else{
        printf("\nLogin Failed. Exiting...\n");
    }
    return 0;
}
```

Output:

Successful Login:

```
D:\CS\SRMS>code.exe

-Login Screen-
Username:admin
Password:********
-Admin Menu-
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete student
6.Logout
Enter Choice:
```

Unsuccessful Login:

```
D:\CS\SRMS>code.exe

-Login Screen-
Username:admin
Password:********
Login failed. Exiting...

D:\CS\SRMS>
```

2. Login Module:

This module is implemented using the **loginSystem()** function, which is called from the main() function when the program starts. Its main purpose is to check the user's username and password against the records stored in credentials.txt. The module uses a helper function, **getHiddenPassword(),** to hide the password while typing for security. It takes the username and password as input and verifies them. If the credentials are correct, it identifies the user's role (Admin, Staff, or Guest) and allows the program to move to the appropriate menu. If the credentials are incorrect, it shows a login failure message. The output of this module is either the user's role for menu access or a message indicating that the login failed.

```c
int loginSystem(){
    char username[20], password[20];
    char fileUser[20], filePass[20], fileRole[20]; //copying credentails.txt

    printf("\n-Login Screen-\n");

    printf("Username:");
    scanf("%s", username);
    printf("Password:");
    getHiddenPassword(password);

    //to open credential file
    //refernce for comparing - username, password
    FILE *fp = fopen(CREDENTIAL_FILE, "r");
    if(!fp){
        printf("Error: credential.txt not found!\n");
        return 0;
    }
    while(fscanf(fp, "%s %s %s", fileUser, filePass, fileRole) == 3){
        if(strcmp(username, fileUser) == 0 && strcmp(password, filePass) == 0){ //password should match with credential password
            strcpy(currentRole, fileRole); //checking admin
            strcpy(currentUser, fileUser); //checking user
            fclose(fp);
            return 1;
        }
    }
    fclose(fp);
    return 0;
}
```

Helper function used:

```c
void getHiddenPassword(char *password){
    int i = 0;
    char ch;

    while(1){
        ch = getch(); // reads character without printing

        if(ch == 13) // ENTER key
            break;

        if(ch == 8 && i > 0){ // Backspace
            i--;
            printf("\b \b");
            continue;
        }

        password[i++] = ch;
        printf("*"); // print star instead of actual character
    }
    password[i] = '\0';
}
```

3. Main Menu Module:

This module is implemented using the **mainMenu()** function, which is called after a successful login. Its purpose is to display the main menu options and direct the user to the appropriate role-specific menu based on their role, such as Admin, Staff, or Guest. The module takes the user's role as input and uses it to determine which menu to show. Depending on the selection, it calls the corresponding menu function. The output of this module is the navigation to the chosen menu, ensuring that users access only the features allowed for their role.

```c
void mainMenu(){
    if(strcmp(currentRole, "ADMIN") == 0){   //if current role is same as ADMIN display adminMenu
        adminMenu();
    }else if(strcmp(currentRole, "STAFF") == 0){   //display staffMenu
        staffMenu();
    }else{
        guestMenu();    //display Menu
    }
}
```

Output:

Admin Menu:

```
D:\CS\SRMS>code.exe

-Login Screen-
Username:admin
Password:********
-Admin Menu-
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete student
6.Logout
Enter Choice:
```

Staff Menu:

```
-Login Screen-
Username:staff
Password:********
-Staff Menu-
1. Display Students
2. Search Student
3. Update Student
4.Logout
Enter Choice:
```

Guest Menu:

```
-Login Screen-
Username:guest
Password:********
-Guest Menu-
1. Display Students
2. Search Student
3.Logout
Enter Choice:
```

4. Admin Menu Module:

This module uses the **adminMenu()** function, called from the main menu when an Admin logs in. It displays options to add, display, search, update, and delete student records, as well as to log out. The module takes the Admin's choice as input and calls the corresponding student operation functions. Its output is the execution of the selected task or returning to the main menu, allowing the Admin to manage all student operations efficiently.

```c
//admin-menu
void adminMenu(){
    int choice;
    do{
        printf("\n-Admin Menu-\n");
        printf("1. Add Student\n");
        printf("2. Display Students\n");
        printf("3. Search Student\n");
        printf("4. Update Student\n");
        printf("5. Delete student\n");
        printf("6.Logout\n");
        printf("Enter Choice:");
        scanf("%d", &choice);

        switch(choice){
            case 1: addStudent(); break;
            case 2: displayStudents(); break;
            case 3: searchStudent(); break;
            case 4: updateStudent(); break;
            case 5: deleteStudent(); break;
            case 6: printf("Logging out...\n"); return;
            default: printf("Invalid choice!\n");
        }
    } while(1);
}
```

Output:

```
-Login Screen-
Username:admin
Password:********
-Admin Menu-
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete student
6.Logout
Enter Choice:1

Enter Roll:124
Enter Name:sasi
Enter marks:88
Student added successfully!

-Admin Menu-
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete student
6.Logout
Enter Choice:2

Roll:   Name    Marks
--------------------------
123     Harini  90.00
124     sasi    88.00
```

5. Staff Menu Module:

This module uses the **staffMenu()** function, called from the main menu when a Staff user logs in. It displays options to display, search, and update student records, as well as to log out. The module takes the Staff user's choice as input and calls the corresponding student operation functions. Its output is the execution of the selected task or returning to the main menu, allowing the Staff to manage student information within their permitted access.

```c
//staff-menu
void staffMenu(){
    int choice;
    do{
        printf("\n-Staff Menu-\n");
        printf("1. Display Students\n");
        printf("2. Search Student\n");
        printf("3. Update Student\n");
        printf("4.Logout\n");
        printf("Enter Choice:");
        scanf("%d", &choice);

        switch(choice){
            case 1: displayStudents(); break;
            case 2: searchStudent(); break;
            case 3: updateStudent(); break;
            case 4: printf("Logging out...\n"); return;
            default: printf("Invalid choice!\n");
        }
    } while(1);
}

```

Output:

```
-Login Screen-
Username:staff
Password:********
-Staff Menu-
1. Display Students
2. Search Student
3. Update Student
4.Logout
Enter Choice:2
Enter roll to search: 125
Record not found.
```

6. Guest Menu Module:

This module uses the **guestMenu()** function, called from the main menu when a Guest user logs in. It displays options to display and search student records, as well as to log out. The module takes the Guest user's choice as input and calls the corresponding student operation functions. Its output is the execution of the selected task or returning to the main menu, allowing the Guest to view student information with limited access.

```c
//guest-menu
void guestMenu(){
    int choice;
    do{
        printf("\n-Guest Menu-\n");
        printf("1. Display Students\n");
        printf("2. Search Student\n");
        printf("3.Logout\n");
        printf("Enter Choice:");
        scanf("%d", &choice);

        switch(choice){
            case 1: displayStudents(); break;
            case 2: searchStudent(); break;
            case 3: printf("Logging out...\n"); return;
            default: printf("Invalid choice!\n");
        }
    } while(1);
}
```

Output:

```
-Login Screen-
Username:guest
Password:********
-Guest Menu-
1. Display Students
2. Search Student
3.Logout
Enter Choice:1

Roll:   Name    Marks
--------------------------
123     Harini  90.00
124     sasi    88.00
```

7. Add Student Module:

This module is implemented using the **addStudent()** function, which is called from the Admin menu. Its purpose is to add new student records to students.txt by taking the student's roll number, name, and marks as input. The output of this module is the successful addition of the student record, allowing the Admin to expand the student database efficiently.

```c
void addStudent(){
    FILE *fp = fopen(STUDENT_FILE, "a"); //append mode
    struct Student st;

    if(!fp){
        printf("Error opening file!\n");
        return;
    }
    printf("\nEnter Roll:");
    scanf("%d", &st.roll);
    printf("Enter Name:");
    scanf("%s", st.name);
    printf("Enter marks:");
    scanf("%f", &st.marks);

    fprintf(fp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    fclose(fp);

    printf("Student added successfully!\n");
}
```

8. Display Students Module:

This module uses the **displayStudents()** function, which can be called from Admin, Staff, or Guest menus. Its purpose is to read all student records from students.txt and display the roll number, name, and marks of each student. This module does not require any input and outputs the complete list of students, allowing users to view all student information easily.

```c
void displayStudents(){
    FILE *fp = fopen(STUDENT_FILE, "r"); // read mode
    struct Student st;

    if(!fp){
        printf("No students record found!\n");
        return;
    }

    printf("\nRoll:\tName\tMarks\n");
    printf("--------------------------\n");

    while(fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3){
        printf("%d\t%s\t%.2f\n", st.roll, st.name, st.marks);
    }

    fclose(fp);
}
```

9. Search Student Module:

The Search Student Module is implemented using the **searchStudent()** function, accessible from Admin, Staff, or Guest menus. It takes a roll number as input and searches for the corresponding student record in students.txt. The module outputs the student's details if found or a "Student not found" message if the record does not exist, helping users locate specific student information.

```c
void searchStudent(){
    FILE *fp = fopen(STUDENT_FILE, "a"); //append mode
    struct Student st;
    int roll, found=0;

    if(!fp){
        printf("Error opening file!\n");
        return;
    }

    printf("Enter roll to search: ");
    scanf("%d", &roll);
    while(fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3){
        if(st.roll == roll){
            printf("Record found:\n");
            printf("Roll: %d\nName: %s\nMarks: %.2f\n", st.roll, st.name, st.marks);
            found = 1;
            break;
        }
    }
    if(!found){
        printf("Record not found.\n");
    }
    fclose(fp);
}
```

10. Update Student Module:

The Update Student Module uses the **updateStudent()** function, called from Admin or Staff menus. Its purpose is to update an existing student's name and marks based on the roll number provided. The module takes the roll number and new details as input, modifies the record in students.txt, and outputs a confirmation message to indicate whether the update was successful.

```c
void updateStudent(){
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *temp = fopen("temp.txt", "w");
    struct Student st;
    int roll, found = 0;

    if(!fp){
        printf("No student record found!\n");
        return;
    }

    printf("Enter roll to update: ");
    scanf("%d", &roll);
```

```c
    while(fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3){
        if(st.roll == roll){
            found = 1;

            printf("Enter new name: ");
            scanf("%s", st.name);

            printf("Enter new marks: ");
            scanf("%f", &st.marks);

            fprintf(temp, "%d %s %.2f\n", st.roll, st.name, st.marks);
        } else {
            fprintf(temp, "%d %s %.2f\n", st.roll, st.name, st.marks);
        }
    }

    fclose(fp);
    fclose(temp);

    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);

    if(found)
        printf("Record updated successfully!\n");
    else
        printf("Record not found!\n");
}
```

11. Delete Student Module:

The Delete Student Module is implemented using the **deleteStudent()** function, accessible from the Admin menu. It takes a roll number as input and deletes the corresponding student record from students.txt. The module outputs a confirmation message after deletion or indicates failure if the record does not exist, allowing the Admin to manage and maintain accurate student records.

```c
void deleteStudent(){
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *temp = fopen("temp.txt", "w");
    struct Student st;
    int roll, found = 0;

    if(!fp){
        printf("No student record found!\n");
        return;
    }

    printf("Enter roll to delete: ");
    scanf("%d", &roll);

    while(fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3){
        if(st.roll == roll){
            found = 1;
        } else {
            fprintf(temp, "%d %s %.2f\n", st.roll, st.name, st.marks);
        }
    }
```

```c
    fclose(fp);
    fclose(temp);

    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);

    if(found)
        printf("Record deleted successfully!\n");
    else
        printf("Record not found!\n");
}
```

Credentials.txt:

credentials.txt

File Edit View

```
admin admin123 ADMIN
staff staff666 STAFF
guest guest111 GUEST
```

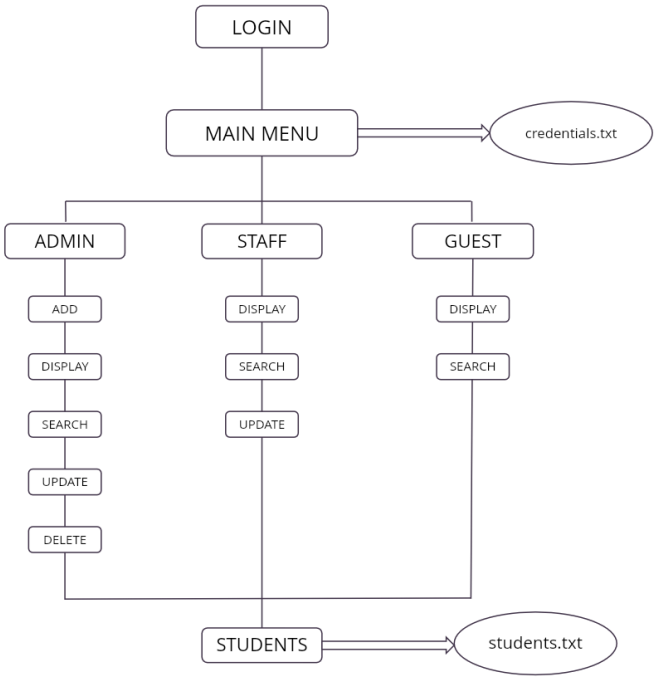
Students.txt:

students.txt

File Edit View

```
123 Harini 90.00
124 Sasi 88.00
```

FLOW CHART DIAGRAM



CONCLUSION

The Student Management System successfully demonstrates the use of C programming concepts such as file handling, structures, and role-based access. The system provides secure login through credentials.txt and manages student data using students.txt, ensuring persistent storage without a database.

With separate menus for Admin, Staff, and Guest users, the program allows controlled access to operations like adding, displaying, searching, updating, and deleting student records. The use of modular functions and temporary files ensures data accuracy and smooth execution.

Overall, this project achieves its objective of creating a simple, efficient, and practical student management application, while also serving as a strong foundation for future enhancements such as GUI interfaces or database integration.

THANK YOU